

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("lab2.ipynb")
```

```
In [ ]: import otter
import pandas as pd
import numpy as np
%matplotlib inline
import otter
import matplotlib.pyplot as plt
grader = otter.Notebook()
```

Pregunta 1.1. En la siguiente celda, asigne el nombre `new_year` al **número mayor** entre los dos números siguientes:

- el **valor absoluto** de $2^6 - 2^{11} - 2^5 - 7$, y
- $5 \times 13 \times 31 + 7$.

Intente utilizar solo una declaración (una línea de código). Asegúrese de verificar su trabajo ejecutando la celda de prueba después.

```
In [ ]: # BEGIN SOLUTION NO PROMPT
new_year=abs(2**6-2**11-2**5-7)
# END SOLUTION

""" # BEGIN PROMPT
new_year=...
""" # END PROMPT
```

```
In [ ]: grader.check("q11")
```

Pregunta 2.1. El módulo `math` también proporciona el nombre `e` para la base del logaritmo natural, que es aproximadamente 2,71. Calcule $e^\pi - \pi$, dándole el nombre `near_twenty`.

Recuerda: ¡También puedes acceder a `pi` desde el módulo `math`!

```
In [1]: from math import *

# BEGIN SOLUTION NO PROMPT
near_twenty = e**pi-pi
# END SOLUTION
```

```
""" # BEGIN PROMPT
near_twenty=...
""" # END PROMPT
```

```
near_twenty
```

```
Out[1]: 19.99909997918947
```

```
In [ ]: grader.check("q21")
```

Pregunta 2.1.1. Calcule el piso de pi usando floor y pi del módulo math. Asigne al resultado el nombre floor_of_pi.

```
In [4]: from math import *

# BEGIN SOLUTION NO PROMPT
floor_of_pi=floor(pi)

# END SOLUTION

""" # BEGIN PROMPT
floor_of_pi=...
""" # END PROMPT

floor_of_pi
```

```
Out[4]: 3
```

```
In [ ]: grader.check("q211")
```

Pregunta 3.2. Utilice num_columns para encontrar el número de columnas en nuestro conjunto de datos de mercados de agricultores.

Asigne el número de columnas a num_farmers_markets_columns.

```
In [9]: import numpy as np
        from datascience import *

farmers_markets = Table.read_table('farmers_markets.csv')

# BEGIN SOLUTION NO PROMPT
```

```

num_farmers_markets_columns=farmers_markets.num_columns

# END SOLUTION

""" # BEGIN PROMPT
num_farmers_markets_columns= ...
""" # END PROMPT
num_farmers_markets_columns

```

Out[9]: 59

In []: grader.check("q32")

Pregunta 3.3. Utilice `select` para crear una tabla con solo el nombre, ciudad, estado, latitud (y) y longitud (x) de cada mercado. Llame a esa nueva tabla “farmers_markets_locations”.

Sugerencia: Asegúrese de ser exacto al usar nombres de columnas con `select`; ¡Compruebe dos veces las mayúsculas!

```

In [24]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

# BEGIN SOLUTION NO PROMPT
farmers_markets_locations=farmers_markets.select('y','x')

# END SOLUTION

""" # BEGIN PROMPT
farmers_markets_locations= ...
""" # END PROMPT
farmers_markets_locations

```

Out[24]:

y	x
44.411	-72.1403
41.3751	-81.7286
42.296	-85.5749
34.8042	-82.8187
37.4956	-94.2746
40.7939	-73.9493
36.1184	-86.7907
40.809	-73.9482
39.7421	-75.5345
38.917	-77.0321
...	(8536 rows omitted)

```
In [ ]: grader.check("q33")
```

Pregunta 3.4. Supongamos que simplemente no desea las columnas `FMID` y `updateTime` en `farmers_markets`. Cree una tabla que sea una copia de `farmers_markets` pero que no incluya esas columnas. Llame a esa tabla “`farmers_markets_ without_fmids`”.

```
In [38]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

         # BEGIN SOLUTION NO PROMPT
         farmers_markets_without_fmids = farmers_markets.drop('FMID', 'updateTime')

         # END SOLUTION

         """ # BEGIN PROMPT
         farmers_markets_without_fmids = ...
         """ # END PROMPT
         farmers_markets_without_fmids
```

```
Out[38]: MarketName | street
         Caledonia Farmers Market Association - Danville | nan
         Stearns Homestead Farmers' Market | 6975 Ridge Road
         100 Mile Market | 507 Harrison St
         106 S. Main Street Farmers Market | 106 S. Main Street
         10th Steet Community Farmers Market | 10th Street and Poplar
         112st Madison Avenue | 112th Madison Avenue
         12 South Farmers Market | 3000 Granny White Pike
         125th Street Fresh Connect Farmers' Market | 163 West 125th Street and Adam Clayton Powe
         12th & Brandywine Urban Farm Market | 12th & Brandywine Streets
         14&U Farmers' Market | 1400 U Street NW
         ... (8536 rows omitted)
```

```
In [ ]: grader.check("q34")
```

Pregunta 3.5. Cree una versión de `farmers_markets_locations` que esté ordenada por **latitud (y)**, con las latitudes más grandes primero. Llámelo “`farmers_markets_locations_by_latitude`”.

```
In [59]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

         # BEGIN SOLUTION NO PROMPT
```

```

farmers_markets_locations=farmers_markets.select('y','x')
farmers_markets_locations_by_latitude=farmers_markets_locations.drop('x')
farmers_markets_locations_by_latitude=farmers_markets_locations_by_latitude.sort('y')

# END SOLUTION

""" # BEGIN PROMPT
farmers_markets_locations_by_latitude= ...
""" # END PROMPT

farmers_markets_locations_by_latitude

```

```

Out[59]: y
17.7099
17.7322
17.7449
17.9686
17.9723
17.9782
17.9792
17.9854
18.0069
18.0089
... (8536 rows omitted)

```

```

In [ ]: grader.check("q35")

```

Pregunta 3.6. Utilice `california_farmers_markets` para crear una tabla llamada `berkeley_markets` que contenga los mercados de agricultores en Berkeley, California.

```

In [84]: import numpy as np
from datascience import *

farmers_markets = Table.read_table('farmers_markets.csv')

# BEGIN SOLUTION NO PROMPT
berkeley_markets=farmers_markets.where('State', are.equal_to('California'))

# END SOLUTION

""" # BEGIN PROMPT
berkeley_markets= ...
""" # END PROMPT

berkeley_markets

```

```

Out[84]: FMID      | MarketName                | street                | city
1005272 | Adelanto Stadium Farmers Market | Hwy 395 & Rancho Drive | Victorville

```

1009019	Alameda Farmers' Market	Haight and Webster St.	Alameda
1008835	Alisal Certified Farmers' Market	632 East Alisal Street	Salinas
1009998	Altadena Farmers' Market	600 W. Palm Street	Altadena
1011421	Alum Rock Village Farmers' Market	57 North White Road	San Jose
1005644	Amador Farmers' Market-- Jackson	Jackson Library/ True Value	Jackson
1005645	Amador Farmers' Market-- Pine Grove	Hwy 88 at Irishtown Rd.	Pine Grove
1005646	Amador Farmers' Market-- Sutter Creek	Eureka St. & Highway 49	Sutter Creek
1002089	Anderson Happy Valley Farmers Market	Happy Valley Rd.	Anderson
1006268	Angels Camp Farmers Market-Fresh Fridays	Utica Park, Utica Lane	Angels Camp
... (745 rows omitted)			

```
In [ ]: grader.check("q36")
```

Pregunta 4.1. Cree una tabla de películas estrenadas entre 2010 y 2015 (inclusive) con calificaciones superiores a 8. La tabla solo debe contener las columnas **Title** and **Rating**, **en ese orden**.

Asigne la tabla al nombre `above_eight`.

Pista: Piensa en los pasos que debes seguir e intenta ordenarlos en un orden que tenga sentido. ¡Siéntete libre de crear tablas intermedias para cada paso, pero asegúrate de asignar a tu tabla final el nombre “`above_eight`”!

```
In [ ]: import numpy as np
        from datascience import *

        imdb = Table.read_table('imdb.csv')

        # BEGIN SOLUTION NO PROMPT
        above_eigh=imdb.where('Rating',are.above_or_equal_to(8)).where('Year',are.between_or_equal_to(2010,2015))

        # END SOLUTION

        """ # BEGIN PROMPT
        above_eigh= ...
        """ # END PROMPT

        above_eigh
```

```
In [ ]: grader.check("q41")
```

Pregunta 4.2. Utilice `num_rows` (y aritmética) para encontrar la *el numero* de películas en el conjunto de datos que se estrenaron entre 1900 y 1999, y la *el numero* de películas en el conjunto de datos que se estrenaron en el año 2000 o después.

Asigne “`num_en_siglo_20`” a el número de películas en el conjunto de datos que se estrenaron entre 1900

y 1999, y “num_en_siglo_21” a el número de películas en el conjunto de datos que se estrenaron en el año 2000 o después.

Pista: El *número* de películas estrenadas en el siglo XX es el *número* de películas estrenadas en el siglo XX, dividida por el *número total* de películas.

```
In [126]: import numpy as np
          from datascience import *

          imdb = Table.read_table('imdb.csv')

          # BEGIN SOLUTION NO PROMPT

          num_movies_in_dataset=imdb.num_rows
          num_in_siglo_20=imdb.where('Year',are.between_or_equal_to(1900, 1999))
          num_in_21st_century=imdb.where('Year',are.above_or_equal_to(2000))

          proportion_in_20th_century=num_in_siglo_20.num_rows/num_movies_in_dataset
          proportion_in_21st_century=num_in_21st_century.num_rows/num_movies_in_dataset

          # END SOLUTION

          """ # BEGIN PROMPT
          num_movies_in_dataset = ...
          num_in_20th_century = ...
          num_in_21st_century = ...
          proportion_in_20th_century = ...
          proportion_in_21st_century = ...
          """ # END PROMPT

          proportion_in_20th_century
          #proportion_in_21st_century
```

```
Out[126]: 0.684
```

```
In [ ]: grader.check("q42")
```

0.1 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

These are some submission instructions.

```
In [ ]: # Save your notebook first, then run this cell to export your submission.  
        grader.export(run_tests=True)
```